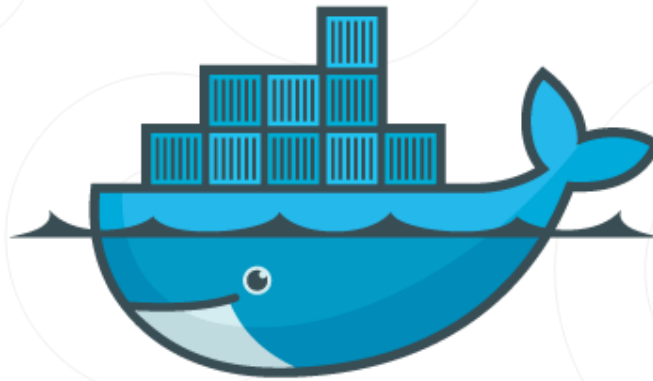


DOCKER

INTRO TO A REVOLUTION

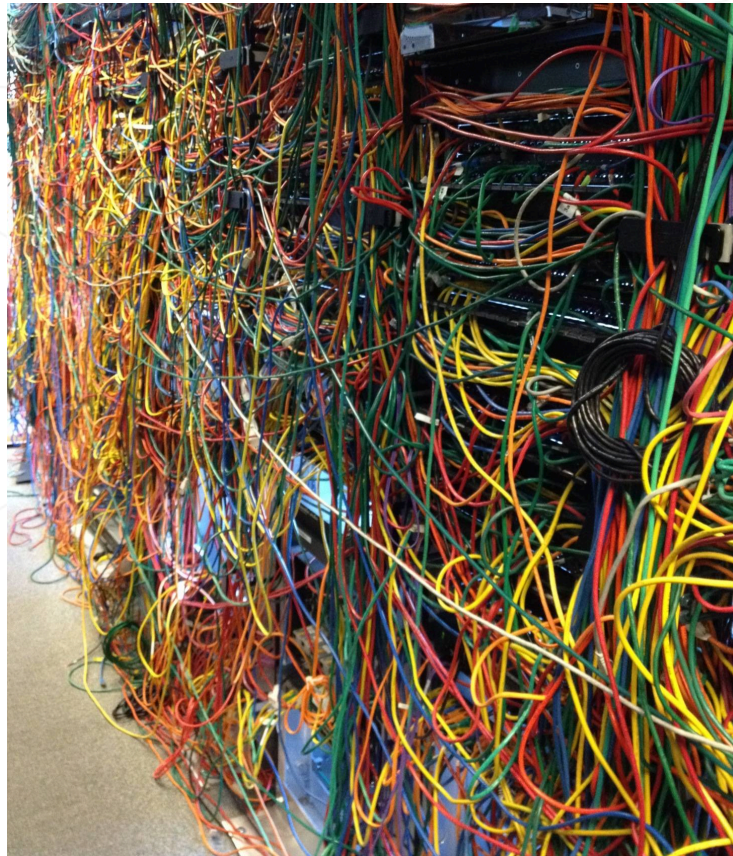
@marekgoldmann

GeeCON, 15th May 2014



docker

APPLICATIONS ARE COMPLEX



http://www.reddit.com/r/pics/comments/nf8la/im_always_tempted_to_switch_two_random_cables_to/

DIFFERENT DEPLOYMENT ENVIRONMENTS



<http://www.shedworking.co.uk/2012/03/nuts-and-bolts-cable-ties.html>

BUT HOW?

UNIFIED DEPLOYMENT FORMAT



docker

A project to create and manage containers.

CONTAINERS?

Yes, a lightweight operating system virtualization.

HOW THIS COMPARES TO A VIRTUAL MACHINE?

It's **completely** different.

COMPARISON

	Virtual Machine (KVM, VMware)	Container (LXC, OpenVZ)
Hardware	Simulated	Uses it (almost) directly
Supported OS'es	Almost any	Only Linux
Space	User space	Kernel space
Separation	Full	Control Groups (cgroups)
Startup time	Seconds to minutes	Miliseconds
Scalability	A few	Sky is the limit (thousands)
Custom kernel	Yes	No
Enterprise features (live migration, etc)	Yes	No
Ease of creation	Moderate	Easy
Time consumption of creation	High	Low
Size	HUGE	Small

WHO'S BEHIND DOCKER?

Docker, Inc. and a **powerful community.**

SPEAKING OF COMMUNITY...

- **GitHub**
 - >800 watchers
 - >11000 stars
 - >1800 forks
 - >500 open issues
 - >80 pull requests
- Over **400 contributors**
 - avg 17 commits / day
 - **90% are external**
- Many meetups all over the world
 - **50** active **meetup** groups

Achieved after **15 months** since first commit to GitHub

GLOSSARY

- **Image:** a (read-only) *template* with filesystem and data
- **Container:** a running *image*
- **Registry:** place where *images* are hosted/shared

MAIN CONCEPTS

- An image can be linked to a **parent** image
- Image not having a parent is called **base** image
- Container has a state, whereas image doesn't
- You can launch multiple containers from an image

HOW DOES IT WORK?

- **cgroups**
 - groups processes in trees, single entity
- **namespaces**
 - process isolation; network, pid, filesystem, etc
- **filesystem**
 - aufs, btrfs, devicemapper

SHOW IT!

CREATING DOCKER IMAGES

It's simple! Use **Dockerfiles**.

DOCKERFILE

A **plain text** file with instructions that **automates** building the image.

DOCKERFILE EXAMPLE

```
# Base on the Fedora 20 image
FROM fedora

# Upgrade the image
RUN yum -y update

# Install WildFly application server
RUN yum -y install wildfly

# Run WildFly after container boot
ENTRYPOINT /usr/share/wildfly/bin/standalone.sh -c standalone-ha.xml -b 0.0.0.0
```

This should be saved as Dockerfile

BUILDING THE IMAGE

`docker build .`

BUILDING THE IMAGE

```
$ docker build .  
Uploading context 2.56 kB  
Uploading context  
Step 0 : FROM fedora  
----> 58394af37342  
  
[SNIP]  
  
Step 3 : ENTRYPOINT /usr/share/wildfly/bin/standalone.sh -c standalone-ha.xml -b 0.0.0  
----> Running in 72f63daffbd1  
----> e91c26099a67  
Removing intermediate container 72f63daffbd1  
Successfully built e91c26099a67
```

We've built image e91c26099a67

MODIFYING THE IMAGE

1. Change the Dockerfile
2. Execute the `build` command once again

IMAGE REPOSITORIES

1. **Private:** for example hosted inside a company
2. **Public:** index.docker.io

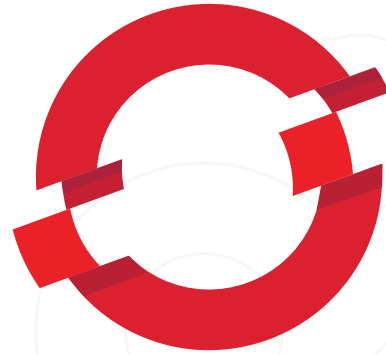
OK, GREAT, **BUT...**

USE-CASES?

- Want your own **PaaS**?
- Automated testing and **CI**
- Deploying **scalable** apps

PAAS

It's (fairly) easy to create a PaaS with Docker
There is even a **tutorial** to create one in 5 minutes...



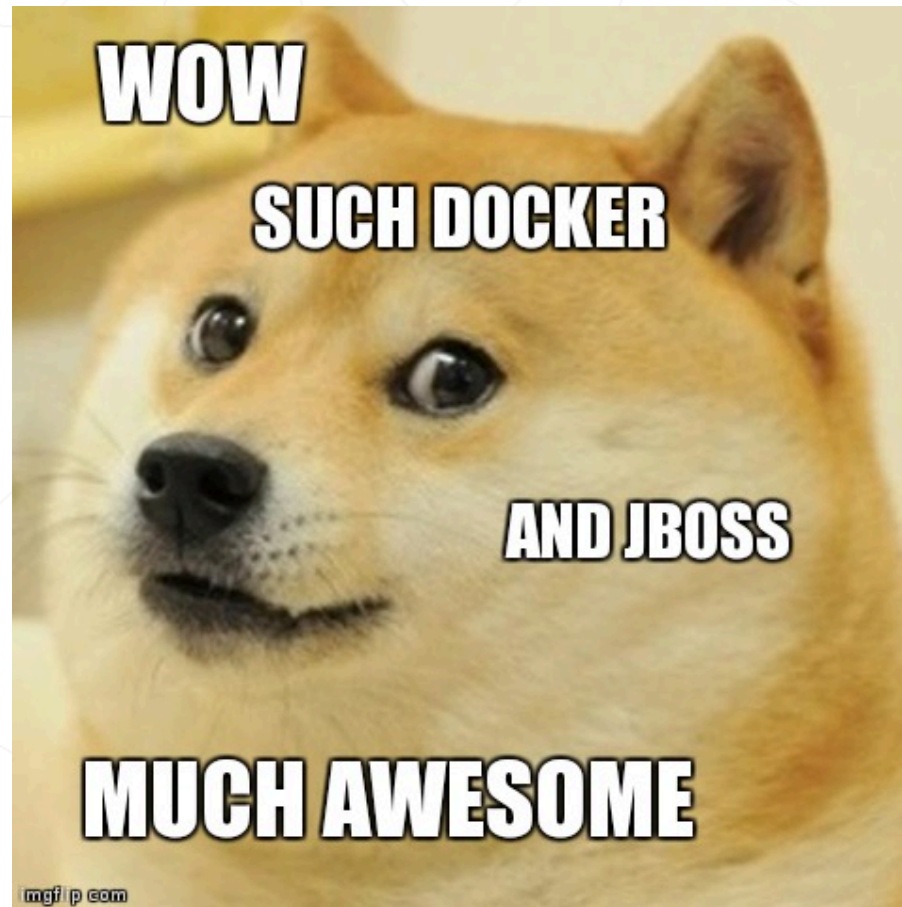
OPENSIFT

CONTINUOUS INTEGRATION

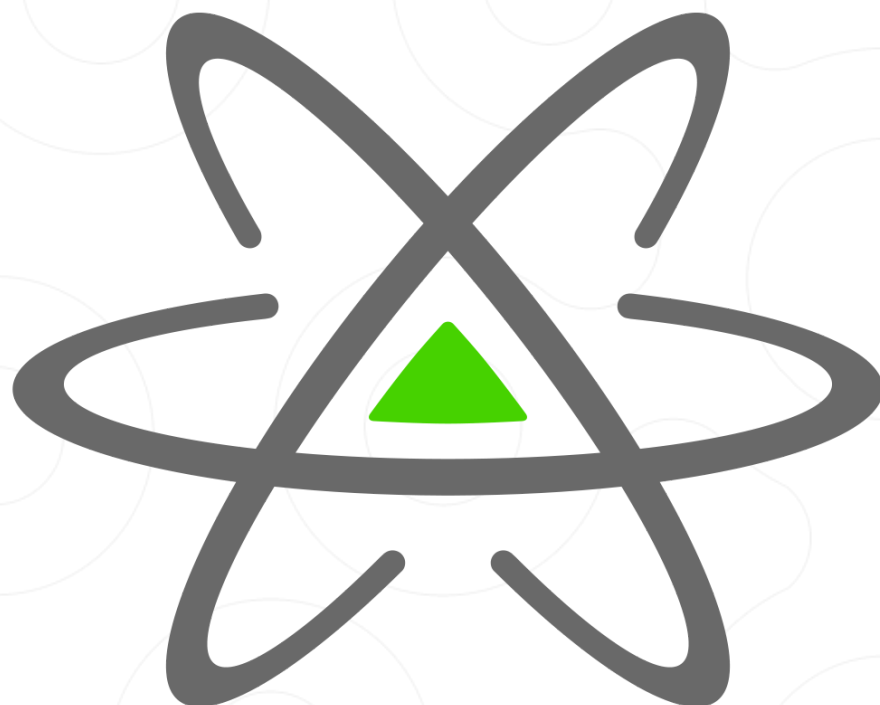
1. You commit
2. CI builds the app
3. CI creates an image with the new app
4. CI starts a container from new image
5. Integration tests are executed on a **real** application
6. **Profit!**

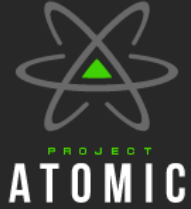
SCALABLE APPS

- **Requirement:** prepare your app to be easily scalable
- Starting a container is fast and cheap
- Extending a (for example WildFly) cluster is easy
- **Profit!**



mgf p com





- 🚀 Get Started
- 📖 Documentation
- ⚙️ Developer Community
- 🔗 On GitHub
- 💬 Ask
- 📖 Blog



Deploy and Manage Your **Docker** Containers.

Project Atomic integrates the tools and patterns of container-based application and service deployment with trusted operating system platforms to deliver an end-to-end hosting architecture that's modern, reliable and secure.

Cloud images are available for download, supporting VirtualBox, QEMU/KVM and OpenStack. Support for bare metal installation is coming.

🚀 GET STARTED →

Trusted Distributions, Atomic Updates

An Atomic Host is a lean operating system designed to run Docker



<http://projectatomic.io>

100 HTTPD SERVERS IN ONE
MINUTE, GO!

THANKS!

(and remember to Docker all the things!)

Slides



Attributions:

<https://www.docker.io/>

